Python WebSocket – Complete Notes (Beginner to Advanced)

1. What is WebSocket?

WebSocket is a communication protocol that provides full-duplex, bidirectional communication between a client and a server over a single persistent TCP connection.

বাংলায়: WebSocket এমন একটি communication protocol যার মাধ্যমে Client এবং Server একই connection ব্যবহার করে যেকোনো সময় একে অপরের কাছে data পাঠাতে পারে।

Key Features

- Full-duplex communication.
- Bidirectional communication.
- Persistent connection.
- Low communication overhead.
- Real-time data transfer.
- Supports text and binary messages.
- Works over TCP.

WebSocket Protocols

```text
ws://example.com/ws/
wss://example.com/ws/
```

- `ws://` = Unencrypted WebSocket.
- `wss://` = Secure WebSocket using TLS.

Always use WSS in production.

---

2. HTTP vs WebSocket

Feature	HTTP	WebSocket
Communication	Request-response	Bidirectional
Connection	Usually request-based	Persistent
Data transfer	Client initiates	Both sides initiate
Server push	Requires additional techniques	Native support
Real-time	Polling/SSE	Yes
Protocol	HTTP	WS/WSS

| Overhead | Repeated request overhead | Lower after handshake |

HTTP Example

```text
Client → Request → Server
Client ← Response ← Server
```

WebSocket Example

```text
Client ↔ Server
Client ↔ Server
Client ↔ Server
```

Explanation: HTTP সাধারণত Client-এর request-এর পর Server response দেয়। WebSocket connection তৈরি হওয়ার পর উভয় পক্ষ স্বাধীনভাবে message পাঠাতে পারে।

---

3. How WebSocket Works

WebSocket communication works through the following steps:

1. Client sends an HTTP request.
2. Client requests protocol upgrade.
3. Server accepts the upgrade.
4. HTTP connection upgrades to WebSocket.
5. Persistent connection is established.
6. Client and Server exchange messages.
7. Either side can close the connection.

WebSocket Handshake

Client request:

```http
GET /ws HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: example-key
```

Sec-WebSocket-Version: 13
```

Server response:

```http
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: ...
```

`101 Switching Protocols` means the server has accepted the protocol upgrade.

---

4. WebSocket Architecture

```text
            CLIENT
              |
              |
         WebSocket Request
              |
              v
            SERVER
              |
         Accept Connection
              |
              v
        Persistent Connection
              |
      ---------------------
      |                   |
      v                   v
   Send Data          Receive Data
      |                   |
      ---------------------
              |
              v
          Close Connection
```

Main Components
```

- Client: Initiates the connection.
- Server: Accepts the connection.
- Connection: Persistent communication channel.
- Message: Data exchanged between client and server.
- Frame: Unit of WebSocket protocol data transfer.

---

5. Python WebSocket Libraries

Library	Purpose
websockets	Async WebSocket server and client
FastAPI	WebSocket API
Django Channels	Django real-time communication
Flask-Sock	Flask WebSocket integration
aiohttp	Async HTTP and WebSocket
Starlette	ASGI WebSocket support

Installation

```bash
pip install websockets
```

FastAPI:

```bash
pip install fastapi uvicorn
```

Django Channels:

```bash
pip install channels channels-redis
```

---

6. Python Async Programming

WebSocket applications commonly use asynchronous programming.

What is Async Programming?

Asynchronous programming allows a program to perform other tasks while waiting for an operation to complete.

বাংলায়: কোনো I/O operation শেষ হওয়ার জন্য পুরো program-কে block না রেখে অন্য কাজ চালিয়ে যাওয়ার পদ্ধতি হলো asynchronous programming।

Important Keywords

Keyword	Meaning
async	Defines asynchronous function
await	Waits for asynchronous operation
asyncio	Python asynchronous programming library
create_task()	Schedules an asynchronous task
gather()	Runs multiple awaitables concurrently

Example

```python
import asyncio

async def say_hello():
    print("Hello")

async def main():
    await say_hello()

asyncio.run(main())
```

asyncio.sleep()

```python
import asyncio

async def main():

    print("Start")

    await asyncio.sleep(2)

    print("End")
```

```
asyncio.run(main())
```

Note: `asyncio.sleep()` does not block the entire event loop.

---

7. Basic WebSocket Server

File: server.py

```python
import asyncio
import websockets

async def handle_client(websocket):

    print("Client connected!")

    async for message in websocket:

        print(f"Received: {message}")

        await websocket.send(
            f"Server received: {message}"
        )

async def main():

    async with websockets.serve(
        handle_client,
        "localhost",
        8765
    ):

        print("WebSocket server running...")

        await asyncio.Future()

asyncio.run(main())
```

Explanation

- `asyncio`: Handles asynchronous operations.
- `websockets.serve()`: Creates a WebSocket server.
- `handle_client()`: Handles an individual client.
- `async for`: Receives incoming messages.
- `websocket.send()`: Sends messages.
- `asyncio.Future()`: Keeps the server running.

---

8. Basic WebSocket Client

File: client.py

```python
import asyncio
import websockets

async def main():

    uri = "ws://localhost:8765"

    async with websockets.connect(uri) as websocket:

        await websocket.send("Hello Server!")

        response = await websocket.recv()

        print("Server:", response)

asyncio.run(main())
```

Run Application

Terminal 1:

```bash
python server.py
```

Terminal 2:

```bash
```

```
python client.py
```

Output:

```text
Server: Server received: Hello Server!
```

---

9. Important WebSocket Methods

Method	Description
websockets.serve()	Creates WebSocket server
websockets.connect()	Connects WebSocket client
websocket.send()	Sends a message
websocket.recv()	Receives a message
websocket.close()	Closes connection
websocket.ping()	Sends ping
websocket.pong()	Sends pong
websocket.wait_closed()	Waits for connection closure

Example

```python
await websocket.send("Hello")

message = await websocket.recv()

await websocket.close()
```

---

10. WebSocket with JSON

JSON is commonly used to exchange structured data.

Server

```python
import asyncio
```

```
import json
import websockets

async def handle_client(websocket):

    async for message in websocket:

        data = json.loads(message)

        username = data["username"]
        text = data["message"]

        response = {
            "username": username,
            "message": text,
            "status": "received"
        }

        await websocket.send(
            json.dumps(response)
        )

async def main():

    async with websockets.serve(
        handle_client,
        "localhost",
        8765
    ):

        await asyncio.Future()

asyncio.run(main())
```

JSON Message

```json
{
    "username": "Faruk",
    "message": "Hello WebSocket"
}
```

JSON Serialization

```python
json.dumps(data)
```

Converts Python object to JSON string.

JSON Deserialization

```python
json.loads(message)
```

Converts JSON string to Python object.

---

11. WebSocket with FastAPI

FastAPI provides built-in WebSocket support through Starlette.

Installation

```bash
pip install fastapi uvicorn
```

Basic Implementation

File: main.py

```python
from fastapi import FastAPI, WebSocket

app = FastAPI()

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):

    await websocket.accept()

    while True:
```

```
        message = await websocket.receive_text()

        await websocket.send_text(
            f"Message received: {message}"
        )
```

Run Server

```bash
uvicorn main:app --reload
```

WebSocket URL:

```text
ws://127.0.0.1:8000/ws
```

Important FastAPI WebSocket Methods

```python
await websocket.accept()

await websocket.receive_text()

await websocket.receive_bytes()

await websocket.receive_json()

await websocket.send_text("Hello")

await websocket.send_bytes(b"Hello")

await websocket.send_json({
    "message": "Hello"
})

await websocket.close()
```

---

12. WebSocket Connection Lifecycle

A WebSocket connection has the following lifecycle:

1. CONNECTING
2. OPEN
3. CLOSING
4. CLOSED

Connection States

State	Description
CONNECTING	Connection is being established
OPEN	Connection is active
CLOSING	Connection is closing
CLOSED	Connection is closed

Connection Lifecycle Example

```python
await websocket.accept()

try:

    while True:

        message = await websocket.receive_text()

        await websocket.send_text(message)

except Exception:

    print("Connection error")

finally:

    await websocket.close()
```

In FastAPI, use `WebSocketDisconnect` for normal disconnect handling.

---

13. WebSocketDisconnect

When a client disconnects, FastAPI raises `WebSocketDisconnect`.

```python
from fastapi import FastAPI, WebSocket
from starlette.websockets import WebSocketDisconnect

app = FastAPI()

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):

    await websocket.accept()

    try:

        while True:

            message = await websocket.receive_text()

            await websocket.send_text(message)

    except WebSocketDisconnect:

        print("Client disconnected")
```

Why is it important?

It helps clean up connection resources and prevent stale connections.

---

14. Connection Manager

A Connection Manager manages multiple connected WebSocket clients.

Why Connection Manager?

- Track active clients.
- Accept connections.
- Remove disconnected clients.
- Broadcast messages.
- Manage client connections.

Implementation

```python
from fastapi import FastAPI, WebSocket
from starlette.websockets import WebSocketDisconnect

app = FastAPI()

class ConnectionManager:

    def __init__(self):

        self.active_connections = []

    async def connect(self, websocket: WebSocket):

        await websocket.accept()

        self.active_connections.append(websocket)

    def disconnect(self, websocket: WebSocket):

        if websocket in self.active_connections:

            self.active_connections.remove(websocket)

    async def send_personal_message(
        self,
        message: str,
        websocket: WebSocket
    ):

        await websocket.send_text(message)

    async def broadcast(self, message: str):

        for connection in self.active_connections:

            await connection.send_text(message)

manager = ConnectionManager()

@app.websocket("/ws")
```

```
async def websocket_endpoint(websocket: WebSocket):

    await manager.connect(websocket)

    try:

        while True:

            message = await websocket.receive_text()

            await manager.broadcast(message)

    except WebSocketDisconnect:

        manager.disconnect(websocket)
```

Important: This in-memory manager works for a single application process. Multiple workers need shared coordination, typically Redis or another messaging system.

---

15. Broadcasting Messages

Broadcasting means sending one message to all connected clients.

Example

```python
async def broadcast(self, message: str):

    for connection in self.active_connections:

        await connection.send_text(message)
```

Use Cases

- Group chat.
- Live notifications.
- Multiplayer games.
- Real-time dashboards.
- Live sports updates.

Personal Message vs Broadcast

Personal Message	Broadcast
One client	Multiple clients
Private communication	Group communication
send_personal_message()	broadcast()

---

16. WebSocket Authentication

Authentication ensures that only authorized users can connect.

Common Authentication Methods

- JWT Authentication.
- Session Authentication.
- OAuth2.
- API Token.

JWT Authentication Concept

1. User logs in.
2. Server generates JWT.
3. Client receives JWT.
4. Client connects to WebSocket.
5. Server validates token.
6. Connection is accepted or rejected.

FastAPI Example

```python
from fastapi import FastAPI, WebSocket, Query

app = FastAPI()

@app.websocket("/ws")
async def websocket_endpoint(
    websocket: WebSocket,
    token: str = Query(...)
):

    if token != "secret-token":
```

```
        await websocket.close(code=1008)

        return

    await websocket.accept()

    while True:

        message = await websocket.receive_text()

        await websocket.send_text(message)
```

This is only a demonstration. Real applications must validate JWT signatures, expiration, issuer, audience and permissions.

---

17. WebSocket Authorization

Authentication and authorization are different.

Authentication: Who are you?

Authorization: What are you allowed to do?

Example:

- User A can access Room 1.
- User B can access Room 2.
- Admin can access both rooms.

Always validate permissions before allowing access to a room or performing an action.

---

18. WebSocket Ping and Pong

Ping and Pong help detect whether a connection is still alive.

Ping

A message used to check connection availability.

Pong

A response to a Ping frame.

Why Heartbeat?

- Detect dead connections.
- Maintain connection health.
- Detect network failures.
- Improve reliability.

The Python `websockets` library supports automatic ping and pong configuration.

```python
async with websockets.serve(
    handle_client,
    "localhost",
    8765,
    ping_interval=20,
    ping_timeout=20
):
    await asyncio.Future()
```

- `ping_interval`: Time between automatic pings.
- `ping_timeout`: Maximum waiting time for pong.

---

19. WebSocket Reconnection

Sometimes connections fail because of:

- Internet disconnection.
- Server restart.
- Network timeout.
- Proxy interruption.

Clients should reconnect when appropriate.

Simple Reconnection Example

```python
```

```
import asyncio
import websockets

async def connect():

    uri = "ws://localhost:8765"

    while True:

        try:

            async with websockets.connect(uri) as websocket:

                print("Connected!")

                while True:

                    message = await websocket.recv()

                    print(message)

        except Exception as error:

            print("Connection failed:", error)

            await asyncio.sleep(5)

asyncio.run(connect())
```

Production applications should use exponential backoff, maximum retry limits, jitter and authentication refresh where necessary.

---

20. Redis Pub/Sub with WebSocket

Redis Pub/Sub allows multiple application instances to exchange messages.

Why Redis?

Consider an application running multiple server instances:

```text
```

```
        CLIENTS
          |
     LOAD BALANCER
       /     \
      /       \
    SERVER 1   SERVER 2
      |         |
      +----REDIS-+
```

Without shared messaging, a message received by Server 1 may not reach clients connected to Server 2.

Redis Pub/Sub solves this coordination problem.

Installation

```bash
pip install redis
```

Redis Connection

```python
import redis.asyncio as redis

client = redis.Redis(
    host="localhost",
    port=6379,
    decode_responses=True
)
```

Publish

```python
await client.publish(
    "chat",
    "Hello Everyone"
)
```

Subscribe

```python
pubsub = client.pubsub()

await pubsub.subscribe("chat")

async for message in pubsub.listen():

    if message["type"] == "message":

        print(message["data"])
```

Important: Redis Pub/Sub is not durable. Disconnected subscribers can miss messages. For persistent event delivery, consider Redis Streams or another message broker.

---

21. WebSocket Rooms

A room allows a group of users to communicate with each other.

Example

```text
Room 1:
   User A
   User B
   User C

Room 2:
   User D
   User E
```

Room Manager Concept

```python
class RoomManager:

    def __init__(self):

        self.rooms = {}

    async def join_room(self, room_id, websocket):
```

```
        await websocket.accept()

        if room_id not in self.rooms:

            self.rooms[room_id] = []

        self.rooms[room_id].append(websocket)

    def leave_room(self, room_id, websocket):

        if room_id in self.rooms:

            if websocket in self.rooms[room_id]:

                self.rooms[room_id].remove(websocket)

    async def send_to_room(self, room_id, message):

        for connection in self.rooms.get(room_id, []):

            await connection.send_text(message)
```

This is a basic example. Production room managers also need disconnect cleanup, authorization and concurrent-send handling.

---

22. WebSocket Message Types

WebSocket supports different types of messages.

Text Message

```python
await websocket.send_text("Hello")
```

Binary Message

```python
await websocket.send_bytes(b"Hello")
```

JSON Message

```python
await websocket.send_json({
    "username": "Faruk",
    "message": "Hello"
})
```

Receive Messages

```python
text = await websocket.receive_text()

binary = await websocket.receive_bytes()

data = await websocket.receive_json()
```

---

23. WebSocket Close Codes

Code	Meaning
1000	Normal closure
1001	Going away
1002	Protocol error
1003	Unsupported data
1006	Abnormal closure (reserved)
1007	Invalid frame payload data
1008	Policy violation
1009	Message too big
1011	Internal server error

Example

```python
await websocket.close(code=1000)
```

---

24. WebSocket Security Best Practices

Security is important in production applications.

Best Practices

1. Always use WSS.
2. Validate authentication tokens.
3. Validate Origin headers.
4. Validate user permissions.
5. Implement rate limiting.
6. Set message size limits.
7. Validate incoming JSON.
8. Handle exceptions properly.
9. Prevent unauthorized room access.
10. Avoid logging sensitive data.
11. Implement connection limits.
12. Protect against denial-of-service attacks.

Message Validation Example

```python
from pydantic import BaseModel, ValidationError

class ChatMessage(BaseModel):

    username: str

    message: str

def validate_message(data):

    try:

        return ChatMessage.model_validate(data)

    except ValidationError:

        return None
```

---

25. WebSocket vs Server-Sent Events (SSE)

Feature	WebSocket	SSE
Communication	Bidirectional	Server to client
Protocol	WebSocket	HTTP
Real-time	Yes	Yes
Client messages	Yes	Separate HTTP request
Reconnection	Application-managed	Built-in browser support
Use case	Chat, gaming	Live feeds, notifications

---

26. WebSocket vs Django Channels

Feature	WebSocket	Django Channels
Type	Protocol	Django framework extension
Real-time communication	Yes	Yes
Django integration	Manual	Integrated
Group communication	Custom	Channel groups
Redis integration	Custom	Channel layer
ASGI	Compatible	Built around ASGI

Django Channels Installation

```bash
pip install channels channels-redis
```

Add to INSTALLED_APPS

```python
INSTALLED_APPS = [
    "channels",
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
]
```

ASGI Configuration

```python
ASGI_APPLICATION = "config.asgi.application"
```

```

---

# 27. WebSocket with Django Channels

### Consumer

A Consumer handles WebSocket connections and events in Django Channels.

File: consumers.py

```python
from channels.generic.websocket import AsyncWebsocketConsumer
import json

class ChatConsumer(AsyncWebsocketConsumer):

    async def connect(self):

        await self.accept()

    async def disconnect(self, close_code):

        print("Disconnected")

    async def receive(self, text_data):

        data = json.loads(text_data)

        message = data["message"]

        await self.send(
            text_data=json.dumps({
                "message": message
            })
        )
```

Routing

File: routing.py

```python
```
```

```
from django.urls import path
from .consumers import ChatConsumer

websocket_urlpatterns = [

    path("ws/chat/", ChatConsumer.as_asgi()),

]
```

ASGI Configuration

```python
import os

from django.core.asgi import get_asgi_application

from channels.routing import ProtocolTypeRouter, URLRouter

from channels.auth import AuthMiddlewareStack

from chat.routing import websocket_urlpatterns

os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "config.settings"
)

django_asgi_app = get_asgi_application()

application = ProtocolTypeRouter({

    "http": django_asgi_app,

    "websocket": AuthMiddlewareStack(

        URLRouter(websocket_urlpatterns)

    ),

})
```

---

28. Django Channels Groups

Groups allow multiple WebSocket clients to receive the same message.

Group Add

```python
await self.channel_layer.group_add(
    "chat_room",
    self.channel_name
)
```

Group Send

```python
await self.channel_layer.group_send(
    "chat_room",
    {
        "type": "chat_message",
        "message": "Hello Everyone"
    }
)
```

Group Discard

```python
await self.channel_layer.group_discard(
    "chat_room",
    self.channel_name
)
```

Event Handler

```python
async def chat_message(self, event):

    await self.send(
        text_data=json.dumps({
            "message": event["message"]
        })
```

```
    )
```

---

29. WebSocket with Redis Channel Layer

Install:

```bash
pip install channels-redis
```

Django settings.py:

```python
CHANNEL_LAYERS = {

    "default": {

        "BACKEND": "channels_redis.core.RedisChannelLayer",

        "CONFIG": {

            "hosts": [
                ("127.0.0.1", 6379)
            ],

        },

    },

}
```

Redis provides shared communication between application workers.

---

30. WebSocket Error Handling

Common problems:

| Error | Possible Cause |

|---|---|
Connection refused	Server not running
Connection closed	Server/client disconnected
Invalid handshake	Incorrect handshake
Timeout	Network or server delay
Authentication failed	Invalid token
403 Forbidden	Origin or permission rejected
404 Not Found	Incorrect endpoint

Exception Handling

```python
from fastapi import WebSocket
from starlette.websockets import WebSocketDisconnect

@app.websocket("/ws")
async def endpoint(websocket: WebSocket):

    await websocket.accept()

    try:

        while True:

            message = await websocket.receive_text()

            await websocket.send_text(message)

    except WebSocketDisconnect:

        print("Client disconnected")

    except Exception as error:

        print("Error:", error)
```

---

31. WebSocket Performance Optimization

1. Use asynchronous programming.
2. Avoid blocking operations.
3. Use Redis for distributed messaging.

4. Limit message sizes.
5. Use efficient JSON serialization.
6. Implement connection limits.
7. Use appropriate heartbeat settings.
8. Avoid unnecessary broadcasting.
9. Monitor active connections.
10. Use load balancing.
11. Handle backpressure.
12. Optimize database queries.

---

32. WebSocket Deployment

For production deployment, consider:

- ASGI server.
- Nginx reverse proxy.
- HTTPS/TLS.
- Redis.
- Load balancer.
- Authentication.
- Monitoring.
- Logging.
- Connection limits.

Example Uvicorn Command

```bash
uvicorn main:app --host 0.0.0.0 --port 8000
```

Nginx Configuration Concept

```nginx
location /ws/ {

    proxy_pass http://127.0.0.1:8000;

    proxy_http_version 1.1;

    proxy_set_header Upgrade $http_upgrade;

    proxy_set_header Connection "upgrade";
```

```
    proxy_set_header Host $host;

    proxy_read_timeout 3600s;

}
```

---

33. Real-World WebSocket Use Cases

1. Real-time Chat Application.
2. Live Notification System.
3. Live Stock Market Dashboard.
4. Online Multiplayer Games.
5. Real-time Location Tracking.
6. Live Delivery Tracking.
7. Collaborative Document Editing.
8. Live Sports Score Updates.
9. Online Customer Support.
10. Real-time Monitoring Dashboard.

---

34. Beginner to Advanced Practice Projects

Beginner Level

- Echo WebSocket Server.
- Simple WebSocket Client.
- Real-time Message Exchange.
- JSON Message Server.
- Basic Notification System.

Intermediate Level

- Real-time Chat Application.
- Multiple Client Broadcasting.
- User Authentication.
- Online User Status.
- Chat Rooms.
- Private Messaging.

Advanced Level

- FastAPI + Redis Chat Application.
- Django Channels Chat Application.
- Group Messaging.
- JWT Authentication.
- Redis Pub/Sub.
- Real-time Notification Backend.
- Multi-worker WebSocket Architecture.

Expert Level

- Distributed WebSocket Architecture.
- Horizontal Scaling.
- Load Balancing.
- Connection Recovery.
- Message Queue Integration.
- Production Monitoring.
- High-Concurrency WebSocket Server.

---

35. Interview Questions and Answers

Q1. What is WebSocket?

WebSocket is a full-duplex communication protocol that provides persistent, bidirectional communication between a client and a server.

Q2. What is the difference between HTTP and WebSocket?

HTTP follows a request-response model, whereas WebSocket supports bidirectional communication through a persistent connection.

Q3. What is a WebSocket handshake?

The initial HTTP Upgrade process that establishes a WebSocket connection.

Q4. What does 101 Switching Protocols mean?

It indicates that the server has accepted the protocol upgrade request.

Q5. What is full-duplex communication?

Both communication parties can send and receive data simultaneously.

Q6. Why use async and await?

They enable asynchronous execution, helping applications handle I/O operations efficiently.

Q7. What is broadcasting?

Sending a single message to multiple connected clients.

Q8. What is a Connection Manager?

A component that manages active WebSocket connections.

Q9. What is WebSocket authentication?

The process of verifying the identity of a client before allowing access.

Q10. What is WebSocket authorization?

The process of checking whether an authenticated client has permission to perform an action.

Q11. What is Redis Pub/Sub?

A messaging mechanism where publishers send messages to channels and subscribers receive messages from those channels.

Q12. What is Django Channels?

A Django extension that supports asynchronous protocols, including WebSockets.

Q13. What is a WebSocket Consumer?

A Django Channels component that handles connection, disconnection and incoming events.

Q14. What is a WebSocket heartbeat?

A periodic ping/pong mechanism used to detect inactive connections.

Q15. What is WSS?

Secure WebSocket communication over TLS.

Q16. What happens when a client disconnects?

The server should detect the disconnect and clean up the client's connection state.

Q17. What are WebSocket rooms?

Logical groups of connected users who can exchange messages with each other.

Q18. How do you scale WebSocket applications?

Use shared messaging infrastructure such as Redis, multiple ASGI workers, load balancing and proper connection management.

Q19. What is the difference between WebSocket and SSE?

WebSocket supports bidirectional communication, while SSE provides server-to-client streaming over HTTP.

Q20. Is WebSocket suitable for real-time chat applications?

Yes. WebSocket is well suited to real-time chat because both clients and servers can send messages over persistent connections.

---

36. Important Terminology

Term	Meaning
WebSocket	Bidirectional communication protocol
Full-duplex	Simultaneous two-way communication
Handshake	Initial connection negotiation
Upgrade	Switch from HTTP to WebSocket
Persistent Connection	Connection remains open
Frame	Protocol-level data unit
Message	Data exchanged
Ping	Connection health check
Pong	Response to Ping
Broadcasting	Send to multiple clients
Consumer	Django Channels event handler
Channel Layer	Communication between consumers
Redis Pub/Sub	Publish-subscribe messaging
ASGI	Asynchronous Server Gateway Interface
WSS	Secure WebSocket
Reconnection	Re-establish a closed connection

| Heartbeat | Periodic connection health check |

---

37. WebSocket Learning Roadmap

Level	Topics
01	HTTP vs WebSocket
02	WebSocket Architecture
03	Handshake
04	Python Async Programming
05	WebSocket Server
06	WebSocket Client
07	JSON Communication
08	FastAPI WebSocket
09	Connection Manager
10	Broadcasting
11	Authentication
12	Authorization
13	Ping/Pong
14	Reconnection
15	WebSocket Rooms
16	Redis Pub/Sub
17	Django Channels
18	Channel Groups
19	Security
20	Production Deployment

---

38. Official Documentation

1. Python WebSockets:
   https://websockets.readthedocs.io/

2. FastAPI WebSockets:
   https://fastapi.tiangolo.com/advanced/websockets/

3. Django Channels:
   https://channels.readthedocs.io/

4. Redis Pub/Sub:
   https://redis.io/docs/latest/develop/pubsub/

5. Python Asyncio:
   https://docs.python.org/3/library/asyncio.html

6. ASGI:
   https://asgi.readthedocs.io/

---

39. Final Summary

WebSocket is an important technology for building real-time applications.

Most Important Concepts

- HTTP vs WebSocket.
- Handshake.
- Full-duplex communication.
- Async and Await.
- WebSocket Server and Client.
- FastAPI WebSocket.
- Connection Manager.
- Broadcasting.
- Authentication and Authorization.
- Ping and Pong.
- Redis Pub/Sub.
- Django Channels.
- WebSocket Security.
- Production Deployment.

Recommended Learning Sequence

Python Async Programming → WebSocket Fundamentals → Python WebSocket → FastAPI WebSocket → Connection Manager → Broadcasting → Authentication → Redis Pub/Sub → Django Channels → Production Deployment.

Practice Goal: Build a complete Real-Time Chat Application using Python, FastAPI, WebSocket, Redis and PostgreSQL.

Additional Django Channels and Advanced WebSocket Notes

> Note: This section expands topics already introduced earlier. Some Bangla text in the supplied source had broken character spacing; that text is preserved where it could not be reconstructed reliably.

40. WebSocket with Django Channels
Django-তে real-time WebSocket application তৈ রি করতে Django Channels ব্য বহা র করা হয়।
What is Django Channels?
Django Channels হলো Django-এর asynchronous communication support বা ড়া নো র একটি extension।
এটি WebSocket, background communication এবং অন্যা ন্য asynchronous protocol পরি চা লনা করতে সা হা য্য করে ।
Installation
```bash
pip install channels
```
Redis channel layer ব্য বহা র করতে :
```bash
pip install channels-redis
```
Project Structure
```text
myproject/
│
├── manage.py
│
├── config/
│ ├── __init__.py
│ ├── settings.py
│ ├── asgi.py
│ └── urls.py
│
└── chat/
├── __init__.py
├── consumers.py
└── routing.py
```
Step 1: settings.py
```python
INSTALLED_APPS = [
"channels",
"chat",
"django.contrib.admin",
"django.contrib.auth",
"django.contrib.contenttypes",
"django.contrib.sessions",
"django.contrib.messages",
"django.contrib.staticfiles",
```

```
]
ASGI_APPLICATION = "config.asgi.application"
```
Step 2: asgi.py
```python
import os
from django.core.asgi import get_asgi_application
from channels.routing import (
ProtocolTypeRouter,
URLRouter
)
from channels.auth import AuthMiddlewareStack
from chat.routing import websocket_urlpatterns
os.environ.setdefault(
"DJANGO_SETTINGS_MODULE",
"config.settings"
)
django_asgi_app = get_asgi_application()
application = ProtocolTypeRouter({
"http": django_asgi_app,
"websocket": AuthMiddlewareStack(
URLRouter(
websocket_urlpatterns
)
),
})
```
Step 3: consumers.py
Consumer হলো Django Channels-এর এমন একটি component, যা connection এবং message পরি চা লনা করে ।
```python
import json
from channels.generic.websocket import (
AsyncWebsocketConsumer
)
class ChatConsumer(AsyncWebsocketConsumer):
async def connect(self):
await self.accept()
await self.send(
text_data=json.dumps({
"message": "Connected successfully"
})
)
async def disconnect(self, close_code):
print("Disconnected:", close_code)
```

```
async def receive(self, text_data):
data = json.loads(text_data)
message = data.get("message", "")
await self.send(
text_data=json.dumps({
"message": message
})
)
```
Step 4: routing.py
```python
from django.urls import path
from .consumers import ChatConsumer
websocket_urlpatterns = [
path(
"ws/chat/",
ChatConsumer.as_asgi()
),
]
```
Step 5: Run Server
Development-এর জন্য Daphne install করতে পা রো :
```bash
pip install daphne
```
```bash
daphne -p 8000 config.asgi:application
```
Important Consumer Methods
Method	Description
`connect()`	Client connect হলে execute হয়
`disconnect()`	Client disconnect হলে execute হয়
`receive()`	Message receive হলে execute হয়
`send()`	Message পা ঠা য়
`accept()`	Connection গ্রহণ করে
`close()`	Connection বন্ধ করে
Sync vs Async Consumer
Sync Consumer
```python
from channels.generic.websocket import (
WebsocketConsumer
)
class MyConsumer(WebsocketConsumer):
```

```
def connect(self):
self.accept()
def receive(self, text_data):
self.send(text_data=text_data)
def disconnect(self, close_code):
pass
```
Async Consumer
```python
from channels.generic.websocket import (
AsyncWebsocketConsumer
)
class MyConsumer(AsyncWebsocketConsumer):
async def connect(self):
await self.accept()
async def receive(self, text_data):
await self.send(text_data=text_data)
async def disconnect(self, close_code):
pass
```
Difference
Sync Consumer	Async Consumer
Synchronous	Asynchronous
Blocking operations হতে পা রে	Non-blocking I/O support করে
Normal Django sync code-এর জন্য সুবি ধা জনক	Async operations-এর জন্য সুবি ধা জনক
---
41. WebSocket Groups and Broadcasting
What is Broadcasting?
Broadcasting হলো একটি message একা ধি ক connected client-এর কা ছে পা ঠা নো ।
Example:
একটি chat room-এ ১০ জন user connected আছে ।
একজন user message পা ঠা লে বাকি user-রা সে ই message পা বে ।
Django Channels Group
একটি group-এর মধ্যে একা ধি ক WebSocket connection রা খা যা য়।
Consumer Example
```python
import json
from channels.generic.websocket import (
AsyncWebsocketConsumer
)
class ChatConsumer(AsyncWebsocketConsumer):
async def connect(self):
self.room_group_name = "chat_room"
```

```
await self.channel_layer.group_add(
self.room_group_name,
self.channel_name
)
await self.accept()
async def disconnect(self, close_code):
await self.channel_layer.group_discard(
self.room_group_name,
self.channel_name
)
async def receive(self, text_data):
data = json.loads(text_data)
message = data["message"]
await self.channel_layer.group_send(
self.room_group_name,
{
"type": "chat_message",
"message": message
}
)
async def chat_message(self, event):
await self.send(
text_data=json.dumps({
"message": event["message"]
})
)
```
Important Methods
group_add()
Client-কে group-এ যুক্ত করে ।
```python
await self.channel_layer.group_add(
group_name,
channel_name
)
```
group_discard()
Client-কে group থে কে সরি য়ে দে য়।
```python
await self.channel_layer.group_discard(
group_name,
channel_name
)
```

group_send()
Group-এর সব member-এর কা ছে event পা ঠা য়।
```python
await self.channel_layer.group_send(
group_name,
{
"type": "chat_message",
"message": "Hello"
}
)
```
Channel Layer Event Type
```python
{
"type": "chat_message"
}
```
এখা নে `type` হলো handler method-এর না ম নি র্ধা রণ করে ।
`chat_message` type-এর জন্য `chat_message()` method execute হবে ।
---
42. Redis with WebSocket
What is Redis?
Redis হলো একটি in-memory data store, যা caching, messaging এবং অন্যা ন্য দ্রু ত data operations-এর জন্য ব্য বহা র করা হয়।
Django Channels-এ Redis channel layer হি সে বে ব্য বহা র করা যা য়।
Why Redis?
ধরো , তো মা র application-এ দু টি WebSocket server রয়ে ছে ।
```text
CLIENTS
|
------------
| |
Server 1 Server 2
| |
------------
|
Redis
```
একটি Client Server 1-এর সঙ্গে connected।
অন্য Client Server 2-এর সঙ্গে connected।
Server 1 থে কে পা ঠা নো message Server 2-এর Client-দে র কা ছে পৌঁ ছা তে Redis channel layer সা হা য্য করতে পা রে ।
Install Redis Channel Layer
```bash
pip install channels-redis
```

```
## settings.py
```python
CHANNEL_LAYERS = {
"default": {
"BACKEND": (
"channels_redis.core.RedisChannelLayer"
),
"CONFIG": {
"hosts": [
"redis://127.0.0.1:6379/0"
],
},
},
}
```
Redis Responsibilities
- Channel layer messaging
- Cross-server communication
- Group broadcasting
- Temporary message distribution
Important Note
Redis channel layer permanent message storage-এর বি কল্পনয়।
Chat history, user information বা order information স্থা য়ী ভা বে রা খতে PostgreSQL বা অন্য database ব্য বহা র করতে হবে ।
---
43. WebSocket Authentication
WebSocket connection-এর ক্ষে ত্রে ও authentication প্রয়ো জন।
কা রণ যে কো নো user যে ন অন্য user-এর private message বা protected data access করতে না পা রে ।
Authentication Methods
1. Session Authentication
2. Token Authentication
3. JWT Authentication
4. Custom Authentication Middleware
Django Channels Session Authentication
`AuthMiddlewareStack` ব্য বহা র করলে WebSocket scope-এ session-based authenticated user পা ওয়া যা য়।
Example:
```python
class ChatConsumer(AsyncWebsocketConsumer):
async def connect(self):
user = self.scope["user"]
if user.is_authenticated:
await self.accept()
else:
```
```

```
await self.close()
```
JWT Authentication
JWT ব্য বহা র করলে token validate করতে হবে ।
Concept:
```text
Client
|
| JWT Token
|
WebSocket Server
|
| Validate Token
|
Authenticated User
```
Important: Browser-এর native WebSocket constructor arbitrary custom headers সে ট করার সুযো গ দে য় না ।
তা ই browser client-এর ক্ষে ত্রে secure session cookie, অথবা carefully designed token authentication flow ব্য বহা র করা যা য়।
URL query string-এ token পা ঠা লে server, proxy বা access log-এ token থে কে যে তে পা রে । তা ই এটি এড়ি য়ে চলা বা অত্য ন্ত
সতর্কভা বে পরি চা লনা করা উচি ত।
---
44. WebSocket Security
WebSocket application-এ security খুব গুরু ত্বপূর্ণ।
1. Use WSS
Production-এ encrypted WebSocket ব্য বহা র করো ।
```text
wss://example.com/ws/
```
2. Authentication
প্রতি টি connection-এর user identity যা চা ই করো ।
3. Authorization
User যে room বা resource access করার অনুমতি রাখে শুধু সে টি ই access করতে দা ও।
4. Origin Validation
অপরি চি ত website থে কে WebSocket connection গ্রহণ করা বন্ধ করো ।
Django Channels-এ:
```python
from channels.security.websocket import (
AllowedHostsOriginValidator
)
application = ProtocolTypeRouter({
"http": django_asgi_app,
"websocket": AllowedHostsOriginValidator(
```

AuthMiddlewareStack(
URLRouter(websocket_urlpatterns)
)
),
})
```
## 5. Input Validation
Client থে কে আসা message validate করো ।
```python
message = data.get("message")
if not isinstance(message, str):
return
if len(message) > 1000:
return
```
6. Rate Limiting
একজন user যে ন অতি রি ক্ত message পা ঠি য়ে server overload করতে না পা রে ।
7. Message Size Limit
অতি রি ক্ত বড় message reject করো ।
8. Avoid Sensitive Logging
Password, JWT token বা private message log করা উচি ত নয়।
---
45. Polling vs Long Polling vs SSE vs WebSocket
1. Polling
Client নি র্দি ষ্ট সময় পরপর Server-কে request পা ঠা য়।
```text
Client ---> Server
Client ---> Server
Client ---> Server
```
Example:
```javascript
setInterval(async () => {
const response = await fetch("/api/updates/");
const data = await response.json();
console.log(data);
}, 5000);
```
2. Long Polling
Client request পা ঠা য়।
Server data পা ওয়া পর্যন্ত বা timeout হওয়া পর্যন্ত request ধরে রাখে ।
Response পা ওয়া র পর Client আবার request পা ঠা য়।
3. SSE (Server-Sent Events)
Server থে কে Client-এর দি কে একমুখী real-time event stream পা ঠা নো হয়।
```

Client সা ধা রণ HTTP connection ব্য বহা র করে event গ্রহণ করে ।
4. WebSocket
Client এবং Server উভয় দি ক থে কে message পা ঠা নো যা য়।
Comparison
Feature	Polling	Long Polling	SSE	WebSocket
Direction	Client request	Server response	Server to Client	Both directions
Persistent	No	Request-based	Yes	Yes
Real-time	Limited	Better	Yes	Yes
Browser support	Yes	Yes	Yes	Yes
Best for	Simple updates	Older real-time systems	Notifications	Chat and gaming
---
46. Ping and Pong
Ping এবং Pong WebSocket-এর control frame।
এগুলো connection সচল আছে কি না পরীক্ষা করতে সা হা য্য করে ।
Ping
একটি পক্ষ Ping পা ঠা য়।
Pong
অন্য পক্ষ Pong দি য়ে response দে য়।
```text
Client Server
|------ Ping --------->|
| |
|<----- Pong ----------|
| |
```
Why Needed?
কখনো network disconnect হয়ে যা য়, কি ন্তু application বুঝতে পা রে না ।
Heartbeat mechanism ব্য বহা র করলে dead connection শনা ক্ত করা সহজ হয়।
Note: Browser-এর JavaScript WebSocket API সরা সরি Ping/Pong control frame তৈ রি বা পড়া র interface দে য় না ।
সা ধা রণত browser implementation এগুলো protocol level-এ handle করে ।
---
47. Error Handling and Reconnection
Network problem হলে WebSocket connection disconnect হতে পা রে ।
তা ই reconnect logic থা কা গুরু ত্বপূর্ণ।
Basic Reconnection Example
```javascript
let socket;
function connectWebSocket() {
socket = new WebSocket(
"ws://localhost:8000/ws/"
);
socket.onopen = () => {
```

```
console.log("Connected");
};
socket.onmessage = (event) => {
console.log(event.data);
};
socket.onclose = () => {
console.log("Disconnected");
setTimeout(
connectWebSocket,
3000
);
};
socket.onerror = (error) => {
console.log("Error:", error);
};
}
connectWebSocket();
```
Exponential Backoff
প্রতি বার reconnect fail হলে অপে ক্ষা র সময় বা ড়া নো হয়।
Example:
```text
Attempt 1: 1 second
Attempt 2: 2 seconds
Attempt 3: 4 seconds
Attempt 4: 8 seconds
Attempt 5: 16 seconds
```
Improved Reconnection Example
```javascript
let socket;
let retryDelay = 1000;
const MAX_DELAY = 30000;
function connect() {
socket = new WebSocket(
"wss://example.com/ws/"
);
socket.onopen = () => {
console.log("Connected");
retryDelay = 1000;
};
socket.onmessage = (event) => {
console.log(event.data);
};
```

```
socket.onclose = () => {
const delay =
retryDelay + Math.random() * 1000;
setTimeout(connect, delay);
retryDelay = Math.min(
retryDelay * 2,
MAX_DELAY
);
};
}
connect();
```

Production application-এ intentional close, authentication failure এবং temporary network failure আলা দা ভা বে handle করা উচি ত।

---

48. WebSocket Performance Optimization

অনে ক concurrent connection থা কলে performance গুরু ত্বপূর্ণ হয়ে ওঠে ।

1. Use Async Programming

I/O-heavy application-এ async programming ব্য বহা র করো ।

2. Reduce Message Size

অপ্রয়ো জনী য় বড় JSON message পা ঠি ও না ।

3. Avoid Unnecessary Broadcasting

শুধু প্রয়ো জনী য় user বা group-কে message পা ঠা ও।

4. Use Redis

Multiple server-এর মধ্যে message distribute করতে Redis channel layer ব্য বহা র করতে পা রো ।

5. Limit Connections

প্রতি user বা IP-এর connection সং খ্যা সী মি ত রাখো ।

6. Handle Disconnects

Disconnected client-এর resources release করো ।

7. Use Pagination

পুরো নো chat history একবারে না পা ঠি য়ে pagination ব্য বহা র করো ।

8. Monitor Connections

Active connection, message throughput এবং error rate monitor করো ।

---

49. WebSocket Scaling

ধরো , তো মা র application-এ ১০,০০০ concurrent user রয়ে ছে ।

একটি server দি য়ে সব connection পরি চা লনা করা কঠি ন হতে পা রে ।

তখন multiple server ব্য বহা র করা যা য়।

Architecture

```text
USERS
|
LOAD BALANCER
|
```

```
------------------
| | |
Server 1 Server 2 Server 3
| | |
------------------
|
REDIS
|
DATABASE
```

Important Components

Load Balancer

Incoming connection বি ভি ন্ন server-এ distribute করে ।

Multiple Application Servers

একা ধি ক server connection handle করে ।

Redis

Server-এর মধ্যে event বা message exchange করতে সা হা য্য করে ।

Database

Permanent information সং রক্ষণ করে ।

Sticky Sessions

কি ছু deployment setup-এ একই client-এর connection নি র্দি ষ্ট server-এর সঙ্গে ধরে রা খা র প্রয়ো জন হতে পা রে ।
তবে WebSocket handshake সম্পন্ন হওয়া র পর connection সা ধা রণত সে ই server-এর সঙ্গে ই থা কে ।

---

50. WebSocket Use Cases

1. Chat Application

Real-time messaging।

2. Live Notification

নতুন notification সঙ্গে সঙ্গে পা ঠা নো ।

3. Online Gaming

Real-time player movement এবং game state update।

4. Live Stock Market

Stock price update।

5. Live Tracking

Delivery বা vehicle tracking।

6. Collaborative Editor

একা ধি ক user একই document edit করতে পা রে ।

7. Real-time Dashboard

Live analytics এবং monitoring।

8. Customer Support

Live customer-agent communication।

---

51. WebSocket Best Practices

1. Production-এ WSS ব্য বহা র করো ।
2. Authentication implement করো ।

3. Authorization check করো ।
4. Origin validate করো ।
5. Incoming message validate করো ।
6. Message size limit রাখো ।
7. Rate limiting implement করো ।
8. Proper error handling রাখো ।
9. Reconnection strategy তৈ রি করো ।
10. Dead connection detect করো ।
11. Unnecessary broadcasting এড়ি য়ে চলো ।
12. Sensitive data log করো না ।
13. Database operation carefully handle করো ।
14. Connection cleanup নি শ্চি ত করো ।
15. Monitoring এবং logging implement করো ।
---
52. Common WebSocket Errors
1. Connection Refused
কা রণ:
- Server চা লু নে ই।
- ভুল port ব্য বহা র করা হয়ে ছে ।
- Endpoint ভুল।
সমা ধা ন:
- Server status check করো ।
- URL এবং port check করো ।
2. WebSocket is Closed Before the Connection is Established
কা রণ:
- Server handshake reject করে ছে ।
- Server crash করে ছে ।
- Authentication failure হয়ে ছে ।
সমা ধা ন:
- Server logs check করো ।
- Authentication এবং routing check করো ।
3. 403 Forbidden
কা রণ:
- Origin validation failure।
- Authentication বা permission failure।
সমা ধা ন:
- Allowed hosts check করো ।
- User authentication এবং authorization check করো ।
4. 404 Not Found
কা রণ:
- WebSocket routing path ভুল।
সমা ধা ন:
- URL pattern check করো ।
5. 500 Internal Server Error

কা রণ:
- Server-side exception।
সমা ধা ন:
- Application logs check করো ।
- Consumer বা endpoint code debug করো ।
6. WebSocketDisconnect
কা রণ:
- Client connection বন্ধ করে ছে ।
- Network connection lost হয়ে ছে ।
সমা ধা ন:
- Disconnect event handle করো ।
---
53. WebSocket Interview Questions
Beginner Level
Q1. What is WebSocket?
Answer:
WebSocket is a communication protocol that provides full-duplex communication between a client and server over a persistent connection.
বাং লা য়:
Client এবং Server-এর মধ্যে real-time bidirectional communication তৈ রি করে ।
Q2. What is the difference between HTTP and WebSocket?
Answer:
HTTP follows a request-response model, while WebSocket provides full-duplex communication over a persistent connection.
Q3. What is ws://?
Answer:
It is the WebSocket URI scheme for an unencrypted WebSocket connection.
Q4. What is wss://?
Answer:
It is the secure WebSocket URI scheme that uses TLS encryption.
Q5. What is a WebSocket handshake?
Answer:
It is the initial process used to establish a WebSocket connection.
Q6. What does HTTP 101 mean?
Answer:
101 Switching Protocols indicates that the server accepted the protocol upgrade request.
Q7. What is full-duplex communication?
Answer:
Both client and server can send and receive data independently at the same time.

Intermediate Level
Q8. What is Broadcasting?
Answer:
Broadcasting means sending one message to multiple connected clients.

Q9. What is a WebSocket Group?
Answer:
A group is a logical collection of WebSocket connections that can receive shared events.
Q10. What is Redis Channel Layer?
Answer:
It is a Channels backend that uses Redis to distribute messages and events across application instances.
Q11. What is Django Channels?
Answer:
Django Channels extends Django to support asynchronous protocols such as WebSocket.
Q12. What is a Consumer?
Answer:
A Consumer is a Channels component that handles connection events and incoming messages.
Q13. What is the difference between SyncConsumer and AsyncConsumer?
Answer:
SyncConsumer uses synchronous methods, while AsyncConsumer uses asynchronous methods and supports async I/O.
Q14. What is Ping and Pong?
Answer:
Ping and Pong are WebSocket control frames used to check connection health.

Advanced Level
Q15. How do you scale WebSocket applications?
Answer:
Use multiple application servers, a load balancer, a shared message layer such as Redis, and proper connection management.
Q16. How do you secure WebSocket?
Answer:
Use WSS, authentication, authorization, origin validation, input validation, rate limiting, and message size limits.
Q17. What is Exponential Backoff?
Answer:
It is a reconnection strategy where the waiting time increases after each failed attempt.
Q18. What is the difference between SSE and WebSocket?
Answer:
SSE provides one-way server-to-client event streaming, while WebSocket provides bidirectional communication.
Q19. Does WebSocket use TCP?
Answer:
The traditional WebSocket protocol uses TCP as its transport.
Q20. Can WebSocket send binary data?
Answer:
Yes. WebSocket supports both text and binary messages.
Q21. Is WebSocket stateless?

Answer:
The protocol connection is stateful because it remains open and maintains connection state.
Q22. Does WebSocket replace REST API?
Answer:
No. REST is useful for request-response operations, while WebSocket is useful for continuous real-time communication. They can work together.
---
54. WebSocket Project Ideas
Beginner Projects
1. Echo Server
Features:
- Client message পাঠাবে।
- Server একই message ফিরিয়ে দেবে।
2. Simple Chat Application
Features:
- Send message
- Receive message
- Real-time communication
Intermediate Projects
3. Group Chat Application
Features:
- Multiple rooms
- User join/leave
- Group messaging
- Message history
4. Live Notification System
Features:
- Real-time notification
- User-specific notification
- Unread notification count
5. Live Order Tracking
Features:
- Order status update
- Delivery status
- Real-time location update
Advanced Projects
6. Real-time Collaboration Tool
Features:
- Multiple users
- Live editing
- User presence
- Shared document state
7. Live Trading Dashboard
Features:

- Live market updates
- Real-time charts
- Price notifications
8. Scalable Chat System
Features:
- Django Channels
- Redis
- PostgreSQL
- Authentication
- Multiple chat rooms
- Private messaging
- Message history
- Read receipts
- Online/offline status
---
55. WebSocket Learning Roadmap
Level 1: Fundamentals
- HTTP basics
- Client-Server architecture
- Request-Response
- WebSocket protocol
- Handshake
- ws:// and wss://
Level 2: JavaScript WebSocket
- WebSocket constructor
- onopen
- onmessage
- onerror
- onclose
- send()
- close()
Level 3: Python WebSocket
- websockets package
- Async programming
- Client
- Server
- Send and receive
Level 4: FastAPI WebSocket
- WebSocket endpoint
- accept()
- receive_text()
- send_text()
- WebSocketDisconnect
- Connection manager

Level 5: Django Channels
- ASGI
- Consumer
- Routing
- Authentication
- Groups
- Broadcasting
Level 6: Redis
- Redis fundamentals
- Channel layer
- Group messaging
- Multiple server communication
Level 7: Advanced Concepts
- Security
- Reconnection
- Heartbeat
- Rate limiting
- Scaling
- Load balancing
- Monitoring
Level 8: Build Projects
- Chat application
- Notification system
- Live dashboard
- Scalable messaging system
---
56. Important Commands
Install FastAPI
```bash
pip install fastapi uvicorn
```
Run FastAPI
```bash
uvicorn main:app --reload
```
Install Django Channels
```bash
pip install channels
```
Install Redis Channel Layer
```bash
pip install channels-redis
```
Install Daphne

```bash
pip install daphne
```
Run Django ASGI Application
```bash
daphne -p 8000 config.asgi:application
```
Install Python WebSockets
```bash
pip install websockets
```
---
57. Final Summary
Topic	Key Point
WebSocket	Real-time communication protocol
Full-Duplex	Both sides communicate
Persistent Connection	Connection remains open
Handshake	Initial connection process
HTTP 101	Protocol switching accepted
ws://	Unencrypted WebSocket
wss://	Secure WebSocket
Ping/Pong	Connection health checking
Broadcasting	Send to multiple clients
Group	Collection of connections
FastAPI	Python WebSocket support
Django Channels	Django real-time support
Consumer	Handles WebSocket events
Redis	Cross-server message distribution
ASGI	Asynchronous Server Gateway Interface
SSE	One-way server-to-client events
Scaling	Multiple servers and shared messaging
Most Important Things to Remember
1. WebSocket provides bidirectional communication.
2. WebSocket connection is persistent.
3. WebSocket starts with a handshake.
4. HTTP 101 means protocol switching accepted.
5. `ws://` is unencrypted.
6. `wss://` is encrypted.
7. FastAPI supports WebSocket directly.
8. Django uses Channels for WebSocket support.
9. Redis helps distribute messages between servers.
10. Authentication and authorization are essential.
11. Production applications should use WSS.

12. WebSocket does not replace REST API.

Recommended Documentation

1. MDN WebSocket API:
https://developer.mozilla.org/en-US/docs/Web/API/WebSocket
2. FastAPI WebSocket:
https://fastapi.tiangolo.com/advanced/websockets/
3. Django Channels:
https://channels.readthedocs.io/
4. Python WebSockets:
https://websockets.readthedocs.io/
5. WebSocket Protocol (RFC 6455):
https://www.rfc-editor.org/rfc/rfc6455
6. Redis:
https://redis.io/docs/latest/

End of WebSocket Complete Notes

Practice Goal: Build a real-time Chat Application using Django Channels + Redis + PostgreSQL.